

RELATÓRIO DE RESULTADOS DE TESTES — ESTEIRAS DE CI/CD (GITHUB ACTIONS) 1

1. Identificação e Classificação Teórica dos Testes

- **Sistema Alvo:** Workflows do GitHub Actions nos repositórios Backend, Frontend e Mobile.
- **Tipo de Teste (Quanto à Execução): Teste Dinâmico** (exige a execução direta dos *workflows* através dos *triggers* de push e *Pull Requests*).
- **Tipo de Teste (Quanto ao Acesso ao Código): Teste de Caixa Preta** (focado em validar o comportamento, entradas de commits, geração de artefatos e saídas esperadas sem alterar a instrução interna das ações de terceiros).
- **Nível de Teste: Teste de Integração de Infraestrutura / Sistema** (valida a comunicação entre o GitHub, o repositório de containers Docker Hub e o servidor de e-mail SMTP).
- **Abrangência:** A validação dos padrões dos *workflows* de develop e main aplica-se de forma transversal aos 3 repositórios do ecossistema FaculRide.

2. Métricas dos Testes Executados

Com base no volume total de **20 execuções de teste** realizadas durante a validação da esteira:

- **Total de Execuções/Tentativas:** 20 execuções.
- **Casos de Teste Aprovados (Sucesso):** 12 execuções (60%).
- **Casos de Teste Reprovados (Falhas/Bugs):** 8 execuções (40%).
- **Taxa de Rejeição de Commits Incorretos:** 100% de eficiência (todos os commits fora do padrão *Conventional Commits* foram bloqueados pelo commitlint).
- **Densidade de Defeitos por Módulo da Esteira:**
 - Module *Trigger & Commitlint*: 0 defeitos.
 - Module *Semver & Versioning*: 5 falhas identificadas (cálculo e extração de tags).
 - Module *Docker Build & Push*: 3 falhas identificadas (contexto e tags).
 - Module *Notification*: 0 defeitos.

3. Tabela de Especificação e Resultados dos Casos de Teste

Tabela 1 – Caso de Teste CT-RES-001

Item	Especificação / Detalhe
ID do Teste	CT-RES-001
Funcionalidade	Disparo Automático na branch develop
Tipo de Teste	Dinâmico / Funcional (Caixa Preta)
Nível de Teste	Teste de Integração (GitHub -> Workflow)
Pré-condição	Workflow CI / CD - Develop cadastrado e ativo no repositório
Entrada / Passos	1. Realizar alteração no código na branch develop 2. Executar o comando <code>git push origin develop</code>
Resultado Esperado	O pipeline ser iniciado automaticamente pelo GitHub Actions
Resultado Obtido	Workflow disparado com sucesso nas 3 frentes (Backend, Frontend e Mobile)
Status Final	Aprovado

Tabela 2 – Caso de Teste CT-RES-002

Item	Especificação / Detalhe
ID do Teste	CT-RES-002
Funcionalidade	Disparo Automático na branch main
Tipo de Teste	Dinâmico / Funcional (Caixa Preta)
Nível de Teste	Teste de Integração (GitHub -> Workflow)
Pré-condição	Workflow CI / CD - Production cadastrado e ativo
Entrada / Passos	1. Realizar merge ou commit na branch main 2. Executar o comando <code>git push origin main</code>
Resultado Esperado	O pipeline de produção ser iniciado automaticamente pelo GitHub Actions

Resultado Obtido	Workflow de produção disparado com sucesso em todas as frentes
Status Final	Aprovado

Tabela 3 – Caso de Teste CT-RES-003

Item	Especificação / Detalhe
ID do Teste	CT-RES-003
Funcionalidade	Validação de Mensagem de Commit Padrão
Tipo de Teste	Dinâmico / Estático (Linting)
Nível de Teste	Teste de Sistema (Validação de Convenção)
Pré-condição	Action <code>wagoid/commitlint-github-action</code> configurada no YAML
Entrada / Passos	1. Criar commit seguindo o padrão Conventional Commits (ex: feat(auth): add login) 2. Fazer git push
Resultado Esperado	O commitlint aprovar a mensagem e a esteira prosseguir para as próximas etapas
Resultado Obtido	Commit validado com sucesso e execução da esteira continuada sem erros
Status Final	Aprovado

Tabela 4 – Caso de Teste CT-RES-004

Item	Especificação / Detalhe
ID do Teste	CT-RES-004
Funcionalidade	Bloqueio de Mensagem de Commit Fora do Padrão
Tipo de Teste	Dinâmico / Funcional (Caixa Preta)
Nível de Teste	Teste de Sistema (Bloqueio e Validação)
Pré-condição	Action commitlint ativa no workflow
Entrada / Passos	1. Criar commit fora da convenção (ex: ajustes de bugs no login) 2. Fazer git push

Resultado Esperado	Interromper a execução do pipeline e indicar erro de formatação no log
Resultado Obtido	Commit rejeitado pelo commitlint e build interrompido como esperado
Status Final	Aprovado

Tabela 5 – Caso de Teste CT-RES-005

Item	Especificação / Detalhe
ID do Teste	CT-RES-005
Funcionalidade	Cálculo do Versionamento Automático (Semver)
Tipo de Teste	Dinâmico / Regra de Negócio (Script)
Nível de Teste	Teste de Integração (Git Tags -> Script de Versão)
Pré-condição	Histórico de commits com tags anteriores cadastradas
Entrada / Passos	1. Enviar commits feat ou fix 2. Executar a etapa de cálculo do Semver
Resultado Esperado	Incrementar corretamente a tag de versão (MAJOR, MINOR ou PATCH)
Resultado Obtido	Erro na leitura do log do git e falha no script em Bash para extração de tags
Status Final	Reprovado

Tabela 6 – Caso de Teste CT-RES-006

Item	Especificação / Detalhe
ID do Teste	CT-RES-006
Funcionalidade	Atualização do arquivo package.json
Tipo de Teste	Dinâmico / Funcional
Nível de Teste	Teste de Unidade / Componente (Modificação do Arquivo)
Pré-condição	Arquivo package.json presente na raiz da aplicação Mobile
Entrada / Passos	1. Calcular a nova versão

	2. Gravar o novo valor no JSON e realizar o auto-commit via bot
Resultado Esperado	Arquivo modificado com a nova versão e commit contendo o sufixo [skip ci]
Resultado Obtido	Arquivo alterado, porém herdando a versão incorreta gerada no step de Semver
Status Final	Reprovado

Tabela 7 – Caso de Teste CT-RES-007

Item	Especificação / Detalhe
ID do Teste	CT-RES-007
Funcionalidade	Construção da Imagem Docker (Build)
Tipo de Teste	Dinâmico / Estrutural
Nível de Teste	Teste de Sistema (Ambiente de Compilação do Container)
Pré-condição	Arquivo Dockerfile configurado na raiz da aplicação
Entrada / Passos	Executar a ação docker/build-push-action
Resultado Esperado	Imagem Docker gerada e compilada sem erros de dependência
Resultado Obtido	Falha de contexto no build e erro na instalação de dependências dentro do container
Status Final	Reprovado

Tabela 8 – Caso de Teste CT-RES-008

Item	Especificação / Detalhe
ID do Teste	CT-RES-008
Funcionalidade	Envio da Imagem ao Docker Hub (Push)
Tipo de Teste	Dinâmico / Integração
Nível de Teste	Teste de Integração (GitHub Actions -> Docker Hub)
Pré-condição	Secrets DOCKERHUB_USERNAME e DOCKERHUB_TOKEN cadastradas

Entrada / Passos	Autenticar no registrador e realizar o push da imagem compilada
Resultado Esperado	Imagem disponibilizada no repositório Docker Hub com as tags da versão
Resultado Obtido	O envio foi interrompido devido às falhas ocorridas na etapa anterior de build
Status Final	Reprovado

Tabela 9 – Caso de Teste CT-RES-009

Item	Especificação / Detalhe
ID do Teste	CT-RES-009
Funcionalidade	Notificação de Falha via E-mail (notify_failure)
Tipo de Teste	Dinâmico / Não Funcional (Alerta e Comunicação)
Nível de Teste	Teste de Integração (GitHub Actions -> Servidor SMTP)
Pré-condição	Ocorrer ou induzir erro em qualquer etapa anterior do pipeline
Entrada / Passos	Disparar a ação dawidd6/action-send-mail ao identificar o status failure()
Resultado Esperado	Enviar e-mail de alerta para a equipe com os detalhes e o link do log
Resultado Obtido	E-mail de notificação disparado e recebido com sucesso contendo o link do log
Status Final	Aprovado

4. Análise dos Problemas Encontrados e Recomendações

4.1 Falha no Cálculo das Versões (Semver)

- **Causa Raiz:** O script Shell utilizado para extrair a última tag via `git tag --list` falha quando não encontra tags anteriores ou quando há sufixos de pré-lançamento (-preview).
- **Ação Corretiva Recomendada:** Padronizar a ação cycjimmy/semantic-release-action (como utilizado no Frontend) em todas as frentes, eliminando os scripts de *Bash* manuais.

4.2 Falha na Imagem Docker

- **Causa Raiz:** Erro no argumento context da ação docker/build-push-action e incompatibilidade nos comandos de instalação de dependências do container.
- **Ação Corretiva Recomendada:** Verificar a localização do Dockerfile na raiz dos projetos e assegurar que as etapas de cópia dos arquivos (COPY . .) incluam todas as dependências do ambiente.